

2024 Ex6 实验报告 -- “公钥”概率加密方案使用说明

实验要求

设计一个实用的“公钥”概率加密方案，使用者可以直接用方案来加密消息明文。

要求：

- 密钥生成算法只输出公钥/私钥对，加密算法所需输入的密钥只有公钥，解密算法所需输入的密钥只有私钥
- 以函数的形式实现方案，需要提交的内容包括：密钥生成函数、加密函数、解密函数
- 一并提交使用文档/手册

实验环境

编程语言：C++

编程环境：C++ 14标准

编程IDE：Visual Studio 2022

方案内容

概述

本“公钥”概率加密方案基于 Paillier 算法上设计生成，旨在提供一种安全的加密方式，使得相同的明文每次加密时生成不同的密文，保证消息的机密性和抗攻击能力。本方案类似于其他公钥加密机制，使用公钥加密消息并用私钥解密消息。

加密范围

本“公钥”概率加密方案针对文本，包括数字、大小写字母、符号以及空格。

使用过程

密钥生成

- 选择大素数 p 和 q ：**
随机选择两个足够大的素数 p 和 q ，确保 $p \neq q$ 。
- 计算 n 和 n^2 ：**
计算 $n = p * q$ ，并计算 $n_squared = n * n$ 。
- 选择加密基数 g ：**
通常设置 $g = n + 1$ ，确保其与 n 的幂模运算有良好的数学性质。
- 计算私钥参数：**
计算 $\lambda = \text{lcm}(p-1, q-1)$ ，即 $p-1$ 和 $q-1$ 的最小公倍数。
计算 $\mu = (L(g^\lambda \bmod n_squared))^{-1} \bmod n$ ，这里 $L(u) = (u - 1) / n$ 。
- 得到公钥私钥：**
公钥 (n, g) ，私钥 (λ, μ) 。

加密

输入

- 公钥 (n, g) : 用于加密的公开参数。
- 明文 m : 需要加密的消息。

步骤

1. 随机数生成:

随机选择一个整数 r , 满足 $1 \leq r < n$, 且与 n 互质。

2. 计算密文:

使用公式 $c = (g^m * r^n) \bmod n_squared$ 计算密文 c 。

注: 每次加密时, 随机数 r 的不同使得相同的明文每次产生的密文也不同。

解密

输入

- 私钥 (λ, μ) : 用于解密的私有参数。
- 密文 c : 要解密的加密消息。

步骤

1. 计算中间值:

计算 $c_lambda = c^\lambda \bmod n_squared$ 。

2. 应用 L 函数:

计算 $l_value = L(c_lambda) = (c_lambda - 1) / n$ 。

3. 计算明文:

使用公式 $m = (l_value * \mu) \bmod n$ 得到解密后的明文 m 。

代码实现

功能代码

1. 生成指定位长度的素数, 用于生成加密算法中的大素数 p 和 q 。注意: 使用 `rand()` 生成随机数是简单的随机数生成方法, 但是安全性不高。在实际应用中, 应该使用更安全的随机数生成器。

```
long long GeneratePrime(int bit_length) {  
    long long prime = 1;  
    while (!IsPrime(prime))  
        prime = (1LL << (bit_length - 1)) + rand() % (1LL << (bit_length - 1));  
  
    return prime;  
}
```

2. 生成一个范围内的随机数, 用于生成加密过程中所需的随机数 r , 确保每次加密同样的明文时, 生成的密文是不同的, 实现概率加密。

```

long long GenerateRandom(long long upper_bound) {
    static random_device rd;
    static mt19937_64 gen(rd());
    uniform_int_distribution<long long> dis(1, upper_bound - 1);
    return dis(gen);
}

```

3. 素数检测。

```

bool IsPrime(long long num) {
    if (num <= 1)
        return false;
    if (num <= 3)
        return true;
    if (num % 2 == 0 || num % 3 == 0)
        return false;
    for (long long i = 5; i * i <= num; i += 6)
        if (num % i == 0 || num % (i + 2) == 0)
            return false;

    return true;
}

```

4. LCM 计算两个数 a 和 b 的最小公倍数；GCD 计算两个数 a 和 b 的最大公约数；L_function 计算 Paillier 加密中的 L 函数，定义为 $L(x) = (x - 1) / n$ 。

```

long long LCM(long long a, long long b) {
    return (a / GCD(a, b)) * b;
}

long long GCD(long long a, long long b) {
    while (b != 0) {
        long long temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

long long L_function(long long x, long long n) {
    return (x - 1) / n;
}

```

5. 通过快速幂算法计算 $(base^{exp}) \% mod$ 。

```

long long ModExp(long long base, long long exp, long long mod) {
    long long result = 1;
    while (exp > 0) {
        if (exp % 2 == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}

```

6. 通过扩展欧几里得算法计算 a 在模 n 下的乘法逆元。

```

long long ModInverse(long long a, long long n) {
    long long t = 0, new_t = 1;
    long long r = n, new_r = a;
    while (new_r != 0) {
        long long quotient = r / new_r;
        t = t - quotient * new_t;
        swap(t, new_t);
        r = r - quotient * new_r;
        swap(r, new_r);
    }

    if (r > 1)
        throw runtime_error("a is not invertible");
    if (t < 0)
        t += n;

    return t;
}

```

密钥生成

```

void KeyGen(int bit_length) {
    long long p = GeneratePrime(bit_length / 2);
    long long q = GeneratePrime(bit_length / 2);

    pub_key.n = p * q;
    pub_key.n_squared = pub_key.n * pub_key.n;
    pub_key.g = pub_key.n + 1;

    long long p_minus_1 = p - 1;
    long long q_minus_1 = q - 1;
    priv_key.lambda = LCM(p_minus_1, q_minus_1);

    long long g_lambda = ModExp(pub_key.g, priv_key.lambda, pub_key.n_squared);
    priv_key.mu = ModInverse(L_function(g_lambda, pub_key.n), pub_key.n);
}

```

加密

```
void Encrypt(vector<long long>& ciphertext, const string& plaintext) {
    for (size_t i = 0; i < plaintext.size(); ++i) {
        long long r;
        do {
            r = GenerateRandom(pub_key.n);
        } while (GCD(r, pub_key.n) != 1);

        long long m = static_cast<long long>(plaintext[i]);
        long long g_pow_m = ModExp(pub_key.g, m, pub_key.n_squared);
        long long r_pow_n = ModExp(r, pub_key.n, pub_key.n_squared);

        long long encrypted_char = (g_pow_m * r_pow_n) % pub_key.n_squared;
        ciphertext.push_back(encrypted_char);
    }
}
```

解密

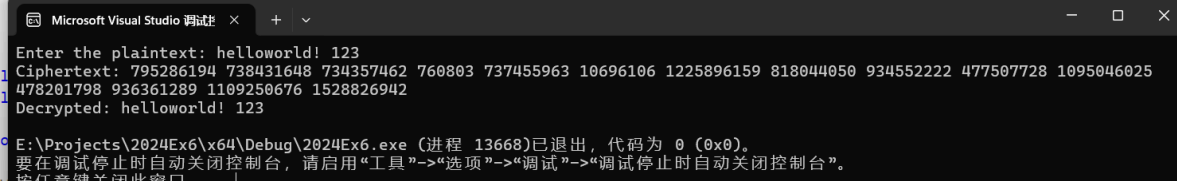
```
void Decrypt(string& plaintext, const vector<long long>& ciphertext) {
    plaintext.clear();

    for (size_t i = 0; i < ciphertext.size(); ++i) {
        long long c_lambda = ModExp(ciphertext[i], priv_key.lambda,
pub_key.n_squared);
        long long l_value = L_function(c_lambda, pub_key.n);
        long long decrypted_char = (l_value * priv_key.mu) % pub_key.n;
        plaintext.push_back(static_cast<char>(decrypted_char));
    }
}
```

结果验证

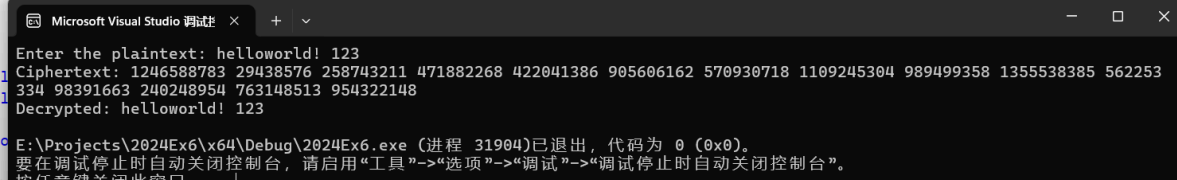
字符串以 helloworld! 123 为例。

第一次:



```
Microsoft Visual Studio 调试
Enter the plaintext: helloworld! 123
Ciphertext: 795286194 738431648 734357462 760803 737455963 10696106 1225896159 818044050 934552222 477507728 1095046025
478201798 936361289 1109250676 1528826942
Decrypted: helloworld! 123
E:\Projects\2024Ex6\x64\Debug\2024Ex6.exe (进程 13668)已退出, 代码为 0 (0x0)。
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口... |
```

第二次:



```
Microsoft Visual Studio 调试
Enter the plaintext: helloworld! 123
Ciphertext: 1246588783 29438576 258743211 471882268 422041386 905606162 570930718 1109245304 989499358 1355538385 562253
334 98391663 240248954 763148513 954322148
Decrypted: helloworld! 123
E:\Projects\2024Ex6\x64\Debug\2024Ex6.exe (进程 31904)已退出, 代码为 0 (0x0)。
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口... |
```

第三次:

```
Microsoft Visual Studio 调试
Enter the plaintext: helloworld! 123
Ciphertext: 1210726661 808306854 1223949231 820615581 776787232 1024739568 1037499397 657271774 79721865 1290575541 1528
094862 1365797399 967729471 1080110338 336121635
Decrypted: helloworld! 123

E:\Projects\2024Ex6\x64\Debug\2024Ex6.exe (进程 54364)已退出，代码为 0 (0x0)。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口...|
```

通过对比三次结果，我们看出对于同一个字符串，加密后的密文均不相同，但解密后的密文均回到原字符串。对此我们认为实现概率加密。

安全性

本加密方案基于大数分解困难性和随机数的概率加密特性来提供安全性，类似于Paillier加密方案。

难题假设：大数因子分解难题

该加密方案的安全性依赖于大整数分解问题， $n = p * q$ ，其中 p 和 q 是大素数。攻击者若想破解密文，需要分解 n ，但大整数分解是一个已知的计算上困难的问题，即只要 n 足够大，分解 n 的计算量对于现有的计算机资源是不可行的，这为方案提供了基础的安全保障。

概率加密的安全性

随机数 r 是用于保证每次加密的结果不同的关键。即使是同一个明文 m ，每次加密时都会生成一个新的随机数 r ，通过计算 $(g^m * r^n) \% n^2$ 来得到密文。由于 r 每次不同，这意味着即使输入相同的明文，生成的密文也不同，从而保证了加密方案的概率安全性。这种性质可以防止攻击者通过比较密文来确定是否有相同的明文输入，抵御重放攻击和选择明文攻击等。

密文隐蔽性

在加密过程中，密文 c 是通过 $(g^m * r^n) \% n_squared$ 计算得到的， g 通常选择为 $n + 1$ ，并且通过同态运算来保持数学上的方便性。 r 是一个与 n 互质的随机数，保证了 r^n 对于不同的 r 取值会均匀分布在模 $n_squared$ 的值域上，从而有效隐藏了原始的明文 m 。

该加密方案基于随机数的均匀分布，即使攻击者知道公钥 n 和 g ，也无法通过密文 c 直接推算出明文 m 。